

Monte Carlo Tree Search for a Simplified Pokémon Battle MDP

Thomas Hoang

Department of Aerospace Engineering Sciences
ASEN 5264: Decision Making Under Uncertainty
Email: thomas.hoang@colorado.edu

Yu Kang Kong

Department of Aerospace Engineering Sciences
ASEN 5264: Decision Making Under Uncertainty
Email: yuko3324@colorado.edu

Abstract—This project models a simplified Pokémon battle as a Markov Decision Process (MDP) and evaluates Monte Carlo Tree Search (MCTS) as an online planning method for action selection. We explore two battle settings: a 1-on-1 battle and a 4-on-4 singles format with switching. In both settings, the opponent is modeled with a stochastic heuristic policy, allowing the battle to remain uncertain while still being interpretable. We evaluate how MCTS performance changes as key search parameters, such as the number of iterations, search depth, and exploration constant, are varied. The results show how increasing the MCTS search budget can improve battle performance and lead to more strategic move and switch decisions.

Index Terms—Markov Decision Process, Monte Carlo Tree Search, online planning, Pokémon battle, stochastic games

I. INTRODUCTION

Pokémon is a game franchise first developed by Nintendo, with *Pokémon Red* released for the Nintendo Game Boy in 1996. The game belongs to the Japanese role-playing game (JRPG) genre, where the player explores a world and participates in turn-based battles. In these battles, a player and an in-game opponent use creatures called Pokémon to deal damage until one side's Pokémon faint.

Each Pokémon has battle related stats, such as health points (HP), attack strength, defensive strength, and speed. HP determines how much damage a Pokémon can take before fainting, while offensive and defensive stats influence how much damage is dealt or received. In a battle, each Pokémon has a set of moves that can be selected on its turn. The simplest moves are damaging moves, which have an associated base power and accuracy. If the move misses, it deals no damage; if it hits, it deals damage based on the move's power and the battle conditions.

Other moves can affect the battle without directly dealing damage. For example, some moves may apply a status condition, such as paralysis, while others may heal the user or increase a stat. Pokémon and their moves also have types, which affect damage through type effectiveness. For example, if a Water-type move hits a Fire-type Pokémon, the move deals twice its normal damage because Water is super effective against Fire. In contrast, if a Fire-type move hits a Water-type Pokémon, the move deals half its normal damage because Fire is not very effective against Water.



Fig. 1. Example Pokémon Showdown battle interface. The UI displays the active Pokémon, move options, team information, and other battle state information that a player uses for decision-making [1].

The goal of a Pokémon battle is to select actions that cause the opponent's team to faint while preventing one's own team from fainting.

Competitive Pokémon also exists as a player-versus-player format, where two human players battle each other instead of playing against a computer controlled opponent. This format has become popular enough to support organized competitions, prize pools, and multiple competitive rulesets across different generations of the game. Because competitive Pokémon involves two players making decisions from a shared battle state, it can be viewed as a Markov game. In this setting, both players select actions, such as attacking or switching, and the next battle state depends on the joint actions of both players.

II. BACKGROUND AND RELATED WORK

Earlier Pokémon games have been heavily studied by fans and researchers through emulation, reverse engineering, and ROM hacking [2]. As a result, many of the mechanics behind the original games have been documented in detail, including the rule-based logic used by computer-controlled opponents during battles. These systems are useful points of comparison

because they show how simple heuristic policies can be used to select actions in a Pokémon battle.

For example, the battle AI in *Pokémon Emerald* can be interpreted as a rule-based decision process. At a high level, the opponent evaluates available Pokémon and moves using hand-coded heuristics, such as type effectiveness, expected damage, and party order [3]. A simplified version of this decision process can be summarized as follows:

- 1) Evaluate type matchups between available moves and opposing Pokémon.
- 2) Select a Pokémon with a favorable matchup when possible.
- 3) Choose the move expected to deal the most effective damage.
- 4) If no clearly favorable option exists, fall back to the current party order or default move selection.

This type of heuristic behavior provides a useful baseline for this project. While rule based policies can perform reasonably well in simple cases, they do not explicitly search over future battle states. In contrast, Monte Carlo Tree Search (MCTS) uses simulated future trajectories to estimate the long-term value of each action, allowing the agent to reason about delayed consequences such as switching, fainting, and future type matchups.

Although the Pokémon battle can be viewed as a Markov game, much of the existing work on Pokémon AI frames the problem through reinforcement learning, self-play, or search-based planning. For example, Sanchez analyzed Pokémon battles as a self-play reinforcement learning problem and studied the challenges of training an agent to play in the Pokémon Showdown environment, a Pokémon Battle Simulation Game. His work identifies several properties that make Pokémon difficult for standard game-playing methods: partial observability, simultaneous or atomic turns, stochastic move outcomes, large branching factors, and costly simulations [4].

One important challenge is that Pokémon decisions are not always cleanly sequential from one player’s perspective. Both players select actions before the turn is resolved, meaning the resulting state depends on both players’ choices. However, this project simplifies the problem by fixing the opponent to a stochastic heuristic policy. Under this assumption, the opponent becomes part of the environment, allowing the battle to be modeled as an MDP and solved using MCTS.

Other work has successfully applied reinforcement learning to Pokémon battle environments. For example, Chen and Lin trained reinforcement learning agents in Pokémon Showdown using battle-state features and player data. In Their work, they frame Pokémon Showdown as an RL environment where the agent observes battle state features and outputs probabilities over legal move indices. They are able to do this because they use real player data in order to train their RL learning algorithm from Pokémon Showdown. Although their agent learned to beat simpler baseline opponents, they concluded that it was still not strong enough to reach human-level play [5].

Some recent work frames competitive Pokémon as a partially observed reinforcement learning problem. For example, Grigsby et al. describe Competitive Pokémon Singles as a game involving imperfect information, long stochastic battles, and opponent adaptation. Rather than modeling the game as a fully observable MDP, they take a Bayesian RL or meta-RL perspective in which the opponent’s team composition and policy are treated as latent variables. Under this view, the agent must use the history of observations, rewards, and both players’ previous actions to infer the opponent’s hidden team and moveset [6].

Because implementing the full Pokémon battle as a Markov game is beyond the scope of this project, we simplify the opponent into a fixed stochastic heuristic policy, allowing the battle to be modeled as a single agent MDP. This simplification is consistent with prior work that treats Pokémon battles as sequential decision-making problems. For example, Rill-García notes that Pokémon can be modeled as an MDP when rewards are defined over turn-based decisions, while also acknowledging that the full game is more accurately described as a Markov game because two opposing agents act toward competing goals [7]. In that work, Q-learning is used as a model-free approach to learn a policy without explicitly modeling the transition function.

MCTS has also been applied directly to simplified Pokémon battle environments. Vlug evaluated MCTS in a simplified 1-vs-1 Pokémon battle setting against several rule-based opponents, studying how the maximum number of MCTS iterations affected performance and runtime [8]. In that work, MCTS was tested using seven Pokémon in 1-vs-1 battle formats, and the results showed that increasing the iteration budget improved performance against rule-based AI. This project builds on the same general idea of using MCTS for Pokémon battle planning, but focuses on a different simplified formulation. We evaluate both a 1-vs-1 battle and a 4-vs-4 singles format with switching actions. The opponent is modeled with a stochastic heuristic policy, and the MCTS agent is evaluated across different search budgets to study how the number of iterations affects win rate, average return, battle length, and switching behavior.

III. PROBLEM FORMULATION

Before formulating the simplified Pokémon battle, we list the assumptions made to simplify the original game:

- Each Pokémon has one type.
- Each Pokémon has a fixed set of moves.
- Damage is based on move power, accuracy, and type effectiveness.
- Special Attack and Special Defense are not in this simplified version
- Flat Attacks and Defense stats do not affect turn, only base move attack
- Player goes first, speed stat only applies to extended battle
- Stats are not Pokémon accurate
- HP is set to 1000 for all Pokémon
- No items and no run action

- Status effects are simplified to a small set of conditions.
- The opponent follows a fixed stochastic heuristic policy.
- The battle ends when all Pokémon on one side have fainted.

Differing from the original proposal, we shifted away from Q-learning and SARSA- λ and instead focused on Monte Carlo Tree Search (MCTS). Q-learning and SARSA- λ require learning values over a large number of state-action pairs, which becomes difficult as the state space grows. In our simplified Pokémon environment, the state space grows quickly due to HP values, statuses, active Pokémon indices, boosts, and switching. However, the action space at each state remains relatively small, since the agent only chooses from move slots and valid switch actions.

For this reason, MCTS was a better fit for the final implementation. Rather than learning a full value table over the entire state space, MCTS performs online planning from the current state using simulated future trajectories. This makes the approach more appropriate for real time decision making in the battle environment, where an agent only needs to choose the best action for the current state rather than solve the entire MDP offline.

We would also like to acknowledge that the original 2-vs-2 reach goal was not fully implemented. After developing the 4-vs-4 singles environment, we decided not to extend the project to a true 2-vs-2 double-battle state space because it would significantly increase both the state space and action space. In particular, a true 2-vs-2 format would require multiple active Pokémon per side, target selection, and joint action resolution, which would mainly increase computational cost without directly supporting the main evaluation goal.

Instead, we redirected the reach-goal effort toward two extensions that were more aligned with the MDP and MCTS analysis: adding different battle mechanics and comparing MCTS against a greedy heuristic baseline. The final implementation adds speed based turn order, paralysis, recovery moves, and attack-boosting moves. These additions make the simplified battle environment more strategic while keeping the problem interpretable enough to evaluate how MCTS performs compared with a short sighted heuristic policy.

We formulate the simplified Pokémon battle as an MDP defined by the tuple

$$\mathcal{M} = (S, A, T, R, \gamma), \quad (1)$$

where S is the state space, A is the action space, T is the transition model, R is the reward function, and γ is the discount factor.

A. State Space

The state is represented as

$$s = (h_p, h_o, z_p, z_o, b_p, b_o, a_p, a_o), \quad (2)$$

where h_p and h_o are the player and opponent HP values, z_p and z_o are the player and opponent status conditions, b_p and b_o are attack boost values, and a_p and a_o are the current active Pokémon on the field who can be targeted by attacks.

B. Action Space

The player action space consists of available move slots:

$$A = \{:\text{move1}, :\text{move2}, :\text{move3}, :\text{move4}, \\ :\text{switch1}, :\text{switch2}, :\text{switch3}, :\text{switch4}\}. \quad (3)$$

Each move slot maps to a move object containing a name, type, category, power, accuracy, and effect. Each Pokémon can have at most four moves, although some Pokémon may have fewer available moves. In the 4-vs-4 singles format, the active Pokémon may also switch to another non-fainted Pokémon in its party. Since the active Pokémon cannot switch to itself, a full party of four allows up to three possible switch actions. More generally, if a party has fewer than four Pokémon, or if some team members have fainted, the number of valid switch actions is reduced.

C. Transition Model

$$T(s' | s, a) = \sum_{a_o \in A_o(s)} \pi_o(a_o | s) P(s' | s, a, a_o), \quad (4)$$

where a is the action selected by the MCTS agent, a_o is the opponent action, $\pi_o(a_o | s)$ is the opponent's stochastic heuristic policy, and $P(s' | s, a, a_o)$ is the probability of transitioning to s' after resolving both actions.

The transition model is implemented as a generative simulator. Given a state s , player action a , and opponent action a_o , the simulator applies the player's move, checks for terminal conditions, samples the opponent's stochastic action, and returns the next state s' . This was done because the MDP formulation produces a large state space, making it impractical to explicitly enumerate a full transition matrix.

For the opponent transition model, the opponent is modeled using a stochastic heuristic policy similar in spirit to the rule based AI used in the Pokémon games. At each turn, the opponent can either use a move or switch to another available Pokémon. The policy assigns a higher total weight to attacking actions than to switching actions, making the opponent more likely to use a move than to switch.

Within the set of possible moves and valid switches, each action is assigned a heuristic weight. Damaging moves receive higher weights when they are super effective against the player's active Pokémon and lower weights when they are not very effective. Switch actions are weighted based on whether the switched-in Pokémon would create a more favorable type matchup. After all action weights are computed, they are normalized into a probability distribution, and the opponent action is sampled from that distribution.

D. Reward Function

The reward function is designed to encourage winning quickly:

$$R(s, a, s') = \begin{cases} 1000, & \text{if all opponent Pokémon faints,} \\ -1000, & \text{if all player Pokémon faints,} \\ 0, & \text{if every Pokémon faints,} \\ -1, & \text{otherwise.} \end{cases} \quad (5)$$

E. Discount Factor

The discount factor is set to

$$\gamma = 0.95. \quad (6)$$

F. Battle Configurations

Two simplified battle configurations were evaluated in this project. The first configuration is a 1-vs-1 battle used as a minimum working example. The second configuration is a 4-vs-4 singles battle, where each side has four Pokémon but only one active Pokémon on the field at a time.

The 1-vs-1 battle uses Pikachu as the MCTS-controlled Pokémon and Squirtle as the opponent. This setup was chosen because the type matchup is easy to interpret: Electric-type moves are super effective against Water-type Pokémon.

TABLE I
1-VS-1 BATTLE CONFIGURATION

Side	Pokémon	Type
Player	Pikachu	Electric
Opponent	Squirtle	Water

TABLE II
EXAMPLE MOVE SET FOR 1-VS-1 BATTLE

Pokémon	Slot	Move	Power	Accuracy
Pikachu	:move1	Tackle	40	1.0
Pikachu	:move2	Thunderbolt	100	1.0
Pikachu	:move3	Nuzzle	40	1.0
Pikachu	:move4	Grass Knot	120	0.8
Squirtle	:move1	Tackle	40	1.0
Squirtle	:move2	Water Gun	60	1.0
Squirtle	:move3	Bite	100	1.0

TABLE III
4-VS-4 SINGLES BATTLE CONFIGURATION

Side	Slot	Pokémon	Type
Player	1	Pikachu	Electric
Player	2	Bulbasaur	Grass
Player	3	Squirtle	Water
Player	4	Charmander	Fire
Opponent	1	Squirtle	Water
Opponent	2	Charmander	Fire
Opponent	3	Bulbasaur	Grass
Opponent	4	Pikachu	Electric

TABLE IV
PLAYER MOVE SET FOR 4-VS-4 BATTLE

Pokémon	Slot	Move	Power	Acc.
Pikachu	:move1	Tackle	40	1.0
Pikachu	:move2	Thunderbolt	100	1.0
Pikachu	:move3	Nuzzle	40	1.0
Pikachu	:move4	Grass Knot	120	0.8
Bulbasaur	:move1	Tackle	40	1.0
Bulbasaur	:move2	Vine Whip	70	1.0
Bulbasaur	:move3	Razor Leaf	90	0.95
Squirtle	:move1	Tackle	40	1.0
Squirtle	:move2	Water Gun	60	1.0
Squirtle	:move4	Bite	100	1.0
Charmander	:move1	Tackle	40	1.0
Charmander	:move2	Ember	60	1.0
Charmander	:move3	Flame Burst	90	0.9
Charmander	:move4	Mud Slap	70	1.0

TABLE V
EXTENDED 4-VS-4 MOVE SET

Pokémon	Slot	Move	Power	Acc.
Bulbasaur	:move1	Recover	40	1.0
Bulbasaur	:move2	Vine Whip	70	1.0
Bulbasaur	:move3	Razor Leaf	90	0.95
Bulbasaur	:move4	Mud Slap	70	1.0
Charmander	:move1	Tackle	40	1.0
Charmander	:move2	Ember	60	1.0
Charmander	:move3	Flame Burst	90	0.9
Charmander	:move4	Swords Dance	0	1.0
Squirtle	:move1	Tackle	40	1.0
Squirtle	:move2	Water Gun	60	1.0
Squirtle	:move3	Bite	100	1.0
Squirtle	:move4	Swords Dance	0	1.0
Pikachu	:move1	Tackle	40	1.0
Pikachu	:move2	Thunderbolt	100	1.0
Pikachu	:move3	Thunder Wave	0	1.0
Pikachu	:move4	Grass Knot	120	0.8

IV. SOLUTION APPROACH

Algorithm 1 MCTS-Based Pokémon Battle Agent

- 1: Initialize battle state s_0
- 2: **while** battle is not terminal **do**
- 3: Run MCTS from current state s
- 4: Select action a with highest estimated value
- 5: Sample opponent action from opponent policy
- 6: Apply both actions using the battle simulator
- 7: Update state $s \leftarrow s'$
- 8: **end while**

The agent uses Monte Carlo Tree Search (MCTS) as an online planning method. At each battle state, MCTS simulates possible future trajectories using the generative battle simulator and selects the action with the highest estimated value. Because the full state space is large, the transition matrix is not explicitly constructed. Instead, next states are sampled through the simulator. Below is a shortened version and general overview of the MCTS algorithm:

Algorithm 2 MCTS Action Selection Shortened

Require: State s , iteration budget N_{iter} , depth d_{max} , exploration constant c

Ensure: Selected action a^*

- 1: **for** $i = 1$ to N_{iter} **do**
- 2: Select actions using UCB:

$$a = \arg \max_{a \in A(s)} \left[Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right]$$

- 3: Expand the first unvisited state
- 4: Simulate a rollout up to depth d_{max}
- 5: Backpropagate the rollout return to update $Q(s, a)$ and $N(s, a)$
- 6: **end for**
- 7: **return** $a^* = \arg \max_{a \in A(s)} Q(s, a)$

UCB is the exploration policy used for MCTS and is as follows:

$$a^* = \arg \max_{a \in A(s)} \left[Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right]. \quad (7)$$

V. RESULTS

A. 1-vs-1 Singles Battle

The 1-vs-1 battle was used as a minimum working example to verify that MCTS could identify strong actions in a simple Pokémon battle. In this setup, Pikachu battles Squirtle, and the expected behavior is that MCTS should favor type-effective moves such as Thunderbolt or Grass Knot over weaker neutral moves such as Tackle.

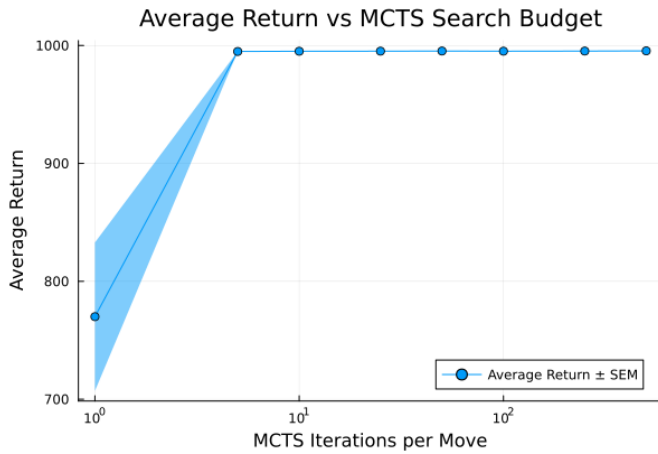


Fig. 2. Average return as a function of MCTS search budget.

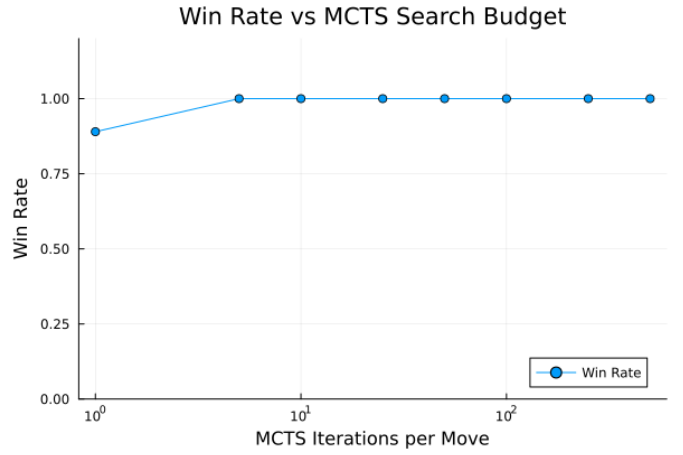


Fig. 3. Win rate as a function of MCTS search budget.

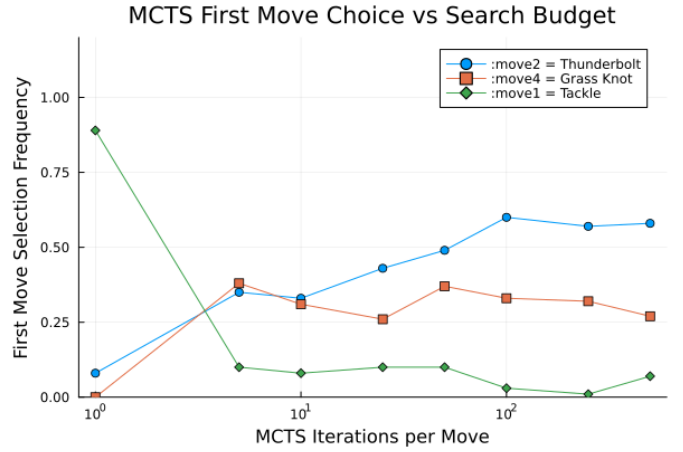


Fig. 4. Frequency of selecting the super-effective opening move as a function of MCTS search budget.

For the 1-vs-1 case, the MCTS parameters were not heavily tuned. The evaluation used 100 episodes, a search depth of 30, and an exploration constant of $c = 1.0$. Because the matchup has a small state and action space, the agent required relatively little tuning to identify strong actions. As the MCTS iteration budget increased, the average return and win rate generally improved. The first-action frequency plot also shows that MCTS favored super effective opening moves over weaker neutral moves. Figure 1 also shows how SEM converges after MCTS iterations give the algorithm a solution to converge to as well.

B. 4-vs-4 Singles Battle

The 4-vs-4 singles battle extends the 1-vs-1 example by adding team HP vectors, active Pokémon indices, and switching actions. This setup evaluates whether MCTS can improve performance in a larger state and action space where the agent must reason about both attacking and switching.

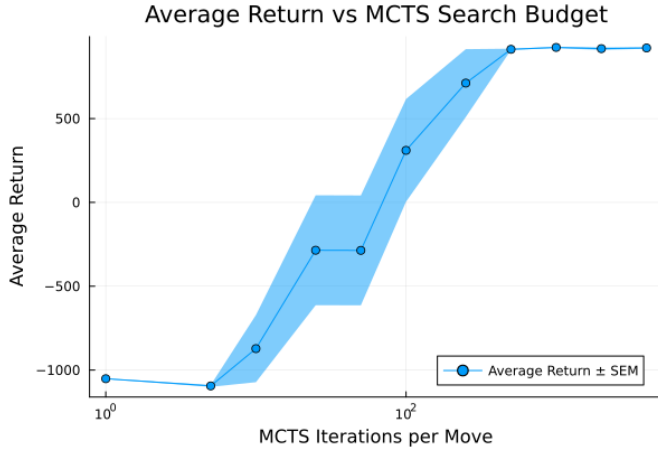


Fig. 5. Average return as a function of MCTS search budget in the 4-vs-4 singles battle.

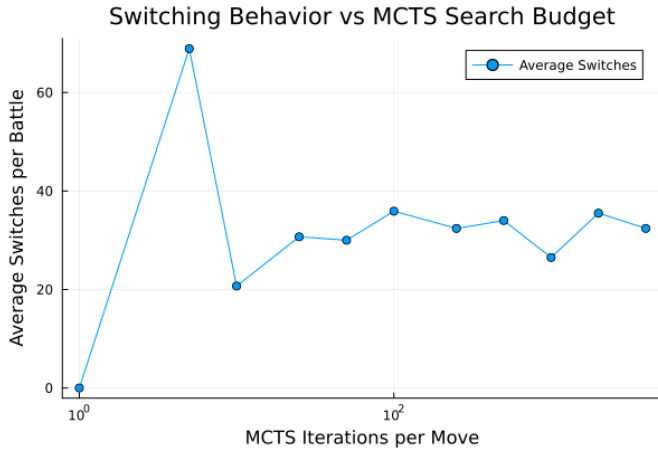


Fig. 6. Average number of switches per battle as a function of MCTS search budget in the 4-vs-4 singles battle.

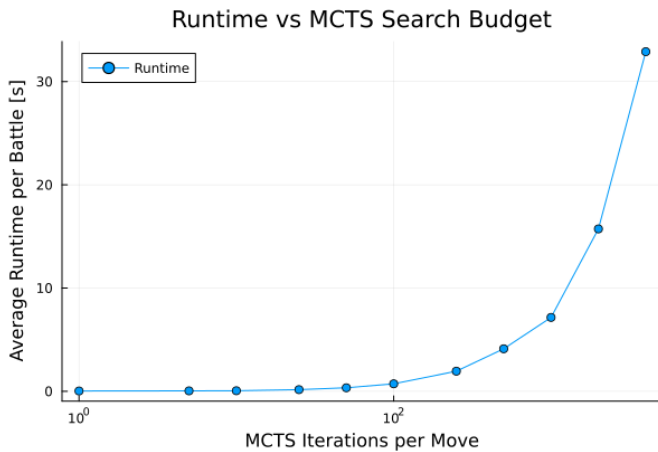


Fig. 7. Average runtime as a function of MCTS search budget in the 4-vs-4 singles battle.

This version of the Pokémon battle required more parameter tuning than the 1-vs-1 case because the 4-vs-4 singles format

has a larger state and action space. The final evaluation used 10 episodes, a search depth of 50, and an exploration constant of $c = 100$. These settings were chosen after testing different parameter values until the MCTS agent consistently reached an average reward above 900 at higher iteration budgets.

The results show that the standard error of the mean (SEM) is largest at lower MCTS iteration budgets, where the agent has not yet converged on a consistent strategy. As the number of MCTS iterations increases, the average reward converges toward the winning reward range, showing that additional search improves decision quality in the larger 4-vs-4 environment.

The runtime plot shows the expected tradeoff: increasing the number of MCTS iterations improves performance but also increases computation time. However, the runtime remained under approximately 10 seconds per move for the tested settings.

C. 4-vs-4 Singles Battle Extended

A greedy heuristic baseline was evaluated to compare MCTS against a short-sighted decision policy. The greedy policy selects actions using immediate battle information, such as expected damage, type effectiveness, healing value, status value, and switching value. Unlike MCTS, it does not simulate future trajectories.

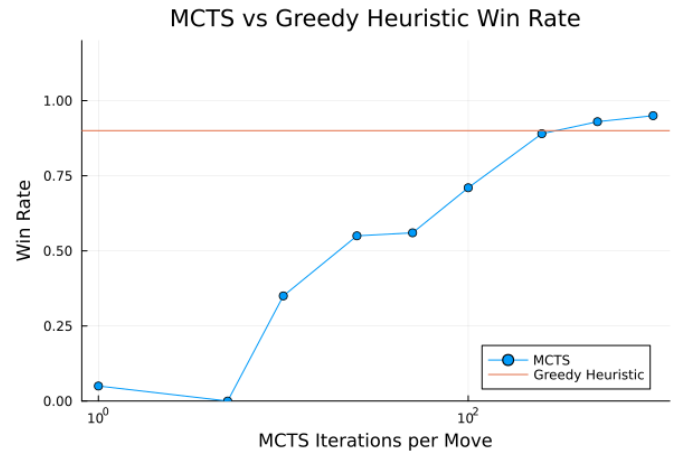


Fig. 8. Win rate comparison between MCTS and the greedy heuristic baseline in the extended 4-vs-4 singles battle. The greedy heuristic is shown as a horizontal baseline because it does not use an MCTS iteration budget.

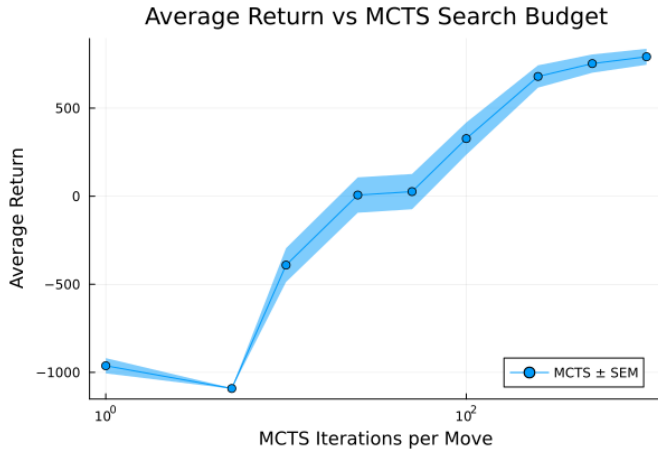


Fig. 9. Average return comparison between MCTS and the greedy heuristic baseline in the extended 4-vs-4 singles battle. The MCTS curve shows how performance changes with search budget, while the greedy heuristic provides a fixed short-sighted baseline.

4v4 Extended value shows the MCTS beating the general Nintendo greedy heuristic policy after more than 1000 iterations. With the average greedy heuristic returning an average of 755.3 in rewards. The MCTS parameters used were 100 episodes, 50 depth, and a 500 exploration constant.

VI. CONCLUSION AND FUTURE WORK

Overall, the 1-vs-1 case served as a minimum working example that verified the MCTS agent could select reasonable actions in a simplified Pokémon battle. The 4-vs-4 singles case then extended this setup to a larger state and action space with switching, showing how MCTS performance changes as the search budget increases. The run time plot also suggests that the tuned MCTS planner can still make decisions within a reasonable time scale for this simplified environment as well.

The comparison and extended 4v4 battle evaluates whether deeper online planning provides an advantage over immediate heuristic decision-making. Since the greedy policy only considers the current state, it can perform well when the best action is obvious, but it may fail to account for delayed effects such as switching, paralysis, recovery, and attack boosts. MCTS can reason over these delayed consequences by simulating future battle trajectories.

Across the evaluated battle scenarios, MCTS performed well as an online planning method for the simplified Pokémon battle environment. As the battle setting became more complex, the average rewards generally decreased, which was expected because the state and action spaces became larger and the agent had to reason over longer battle horizons. However, with sufficient tuning of the MCTS parameters, including the iteration budget, search depth, and exploration constant, the agent was still able to converge toward winning behavior in each scenario.

For future work, this simplified battle environment could be improved by adding more realistic Pokémon mechanics, including speed, move priority, attack, defense, special attack, and special defense stats. These additions would make the

damage and turn order models more representative of the original game.

A longer term goal would be to formulate the battle as a full Markov game. In that setting, both players would act as decision making agents, and methods such as fictitious play or self-play could be used to train against adaptive opponents. This would make the environment more realistic and could eventually allow the agent to compete against human players or stronger learned policies.

VII. CONTRIBUTIONS AND RELEASE

Thomas Hoang contributed to creating the Pokémon battle environment, state representation, transition simulator, reward function, move logic, type effectiveness, 4-vs-4 switching extension, and MCTS plotting.

Yu Kang Kong contributed to creating the mathematical formulation, MCTS explanation, report structure, and analysis of the experimental results.

The authors grant permission for this report to be posted publicly.

The core MDP environment, transition simulator, reward function, and evaluation scripts were implemented by the authors. The MCTS algorithm was used off-the-shelf through the Julia MCTS/POMDPs libraries.

REFERENCES

- [1] M. Kimura-Thollander, “Everyday UI: Pokémon Showdown,” <https://medium.com/@minnakt/everyday-ui-pokemon-showdown-79eea05bae1>, 2019, accessed: 2026-04-29.
- [2] https://docs.google.com/document/d/1CzTi0DbwdOX2cvQN9yldL7zLXfKQtBy_UWuxccpJnAM/edit?pli=1&tab=t.0#heading=h.x2iqofawg2kx, accessed: 2026-04-28.
- [3] https://docs.google.com/document/d/13E61Jj4Kwhly3ZKgLjPY-uWKWklfBR_zUlhZHUZqik/edit?tab=t.0, accessed: 2026-04-28.
- [4] M. L. Sanchez, “Learning Complex Games Through Self Play: Pokémon Battles,” <https://upcommons.upc.edu/server/api/core/bitstreams/f3815b36-f3cf-49c4-9053-07d96442f279/content>, 2018, accessed: 2026-04-29.
- [5] K. Chen and E. Lin, “Learning to Play Pokémon Showdown with Reinforcement Learning,” https://cs230.stanford.edu/projects_fall_2018/reports/12447633.pdf, 2018, accessed: 2026-04-29.
- [6] J. Grigsby, Y. Xie, J. Sasek, S. Zheng, and Y. Zhu, “Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers,” *Reinforcement Learning Journal*, 2025, accessed: 2026-04-29.
- [7] R. Rill-García, “Reinforcement Learning for a Turn-Based Small Scale Attrition Game,” 2018, accessed: 2026-04-29.
- [8] C. A. Vlуг, “Implementing Monte Carlo Tree Search in a Pokémon Battle Based Environment,” <https://arno.uvt.nl/show.cgi?fid=170059>, 2023, bachelor’s thesis, Tilburg University. Accessed: 2026-04-29.